

NewPipe

Maintenance Report

By

The Neon Tetra

Consisting of

Chandler Guthrie,

Jaeger Nelson,

Evan Glasscock,

Steven Bennett

1. Change Requests.....	7
1.1. Change Request Template (Repo) {Issue} [Type].....	7
1.1.1. Description Of Changes.....	7
1.1.2. Pre/Post-Factoring Activities.....	7
1.1.3. Bug Localization.....	7
1.1.4. Change Impact Analysis.....	7
1.1.5. Unit/Regression Testing Activities.....	7
1.1.6. Commit Link.....	7
1.1.7. Code Review.....	7
1.2. Extracting Section Lists From Channel Tab Feature (NewPipe Extractor) {Self-Created} [Enhancement].....	7
1.2.1. Description Of Changes.....	7
1.2.2. Pre/Post-Factoring Activities.....	7
1.2.3. Change Impact Analysis.....	7
1.2.4. Unit/Regression Testing Activities.....	8
1.2.5. Commit Link.....	8
1.3. Extracting Featured Channels (NewPipe Extractor) {1454} [Feature].....	8
1.3.1. Description Of Changes.....	8
1.3.2. Pre/Post-Factoring Activities.....	8
1.3.3. Change Impact Analysis.....	8
1.3.4. Unit/Regression Testing Activities.....	9
1.3.5. Commit Link.....	9
1.4. Exiting Channel Group Returns To Main page (NewPipe) {13221} [Bug].....	9
1.4.1. Description Of Changes.....	9
1.4.2. Pre/Post-Factoring Activities.....	9
1.4.3. Bug Localization.....	9
1.4.4. Change Impact Analysis.....	10
1.4.5. Commit Link.....	10
1.5. Add "Stop after current song ends" playback option (NewPipe) {#12726} [Feature].....	11
1.5.1. Description Of Changes.....	11
The application lacked a feature to play the current track and stop immediately afterwards without clearing the rest of the queue. To implement this, a boolean flag stopAfterCurrent was added to the PlayQueue class, serving as the source of truth for the user's intent. A new menu item was added to menu_play_queue.xml to allow users to toggle this state. The Player.java controller logic was modified to intercept the DISCONTINUITY_REASON_AUTO_TRANSITION event. If the flag is set to true, the player pauses execution and resets the flag, rather than proceeding to the next track...	11
1.5.2. Pre/Post-Factoring Activities.....	11
Initial testing involved hardcoding the stopAfterCurrent flag to true within the PlayQueue constructor to verify the player interception logic before building the UI. It was discovered that NewPipe instantiates specific subclasses (like SinglePlayQueue), so the hardcoded check was moved to Player.initPlayback() for verification. Post-factoring involved	

removing these debug overrides and implementing the dynamic menu toggle in PlayQueueActivity.java.....	11
1.5.3. Change Impact Analysis.....	11
The risk is minimal as the default state of the boolean is false, preserving the existing autoplay behavior for all users unless they explicitly opt-in via the menu. The change touches PlayQueue.java (Model), Player.java (Controller), and PlayQueueActivity.java (View), but adheres to the existing architecture.....	11
1.5.4. Unit/Regression Testing Activities.....	11
Manual verification was performed using the Android Logcat with a custom debug tag (481_DEBUG).....	11
Test Case 1: Enable toggle -> Allow song to finish -> Result: Player paused, Toast notification displayed "Stopping after this song.".....	11
Test Case 2: Enable toggle -> Disable toggle -> Allow song to finish -> Result: Player auto-transitioned to the next track.....	12
Test Case 3: App restart -> Result: Flag successfully reset to false (due to transient keyword), preventing accidental stops in future sessions.....	12
1.5.5. 1.5.5. Commit Link.....	12
https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/884e1971d2950ec9ef0e4b9a549128d7d82d98f7	12
1.6. Playlist isn't queued if the first video is unavailable (NewPipe) {#13226} [Bug].....	12
1.6.1. Description Of Changes.....	12
A "Logic Fail" occurred where an entire playlist would fail to queue if the very first video (Index 0) was unavailable (e.g., region-locked). The UI would display a generic "Content Unavailable" error screen and abandon the queue creation. The fix involved modifying VideoDetailFragment.java. The openMainPlayer() method contained a guard clause that prevented the Player Service from starting if the metadata for the first video was null. This was bypassed to allow the player to launch if a valid queue (PlayQueue) existed. Additionally, the runWorker() error handler was updated to log the failure but force a call to openVideoPlayerAutoFullscreen(), handing control to the backend MediaSourceManager.....	12
1.6.2. Pre/Post-Factoring Activities.....	12
Pre-Factoring: Investigated MediaSourceManager and PlaylistAppendTask assuming the issue was a backend failure. Post-Factoring: Confirmed that the backend logic for skipping bad videos already existed; the UI was simply preventing it from executing.....	12
1.6.3. Bug Localization.....	12
The bug was localized by reproducing the crash using the "Animal Crossing New Horizons OST" playlist, where the first video is unavailable. Logcat showed repeated calls to showSnackBarError() originating from VideoDetailFragment. Breakpoints revealed that openMainPlayer was never being called because the UI fragment treated the single video failure as a fatal error for the entire playlist context.....	13
1.6.4. Change Impact Analysis.....	13
This change improves application robustness. It relies on the existing backend logic in MediaSourceManager to scrub unavailable items. There is a minor impact on user experience where the player may briefly show a "Loading" state for the failed video before skipping to the next valid index, but this is preferable to a total failure.....	13
1.6.5. Commit Link.....	13

https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/c279f528a85145ef27b08ff82ca88cef719bd5e0	13
1.7. Allow upside-down portrait mode (reverse portrait) for video playback (NewPipe) {#12779} [Feature].....	14
1.7.1. Description Of Changes.....	14
The application previously forced a strict "Portrait" orientation or "Landscape" orientation, making it difficult to watch vertical videos while charging the device (which usually requires the phone to be held upside-down). The fix involved updating the AndroidManifest.xml file. The android:screenOrientation attribute for the main activity was changed to "fullSensor". This instructs the Android OS to respect all four orientation degrees (0°, 90°, 180°, 270°) based on the accelerometer, enabling "Reverse Portrait" mode.....	14
1.7.2. Pre/Post-Factoring Activities Pre-Factoring:.....	14
Tested on an Android 16 Emulator. Rotating the device 180 degrees resulted in either no change (UI upside down relative to user) or a forced rotation to Landscape. Post-Factoring: Verified that fullSensor correctly interprets the 180-degree rotation and flips the UI while maintaining the vertical layout.....	14
1.7.3. Change Impact Analysis.....	14
This is a configuration-only change. It has zero impact on codebase logic or compilation time. The only potential side effect is if a user's device accelerometer is overly sensitive, it might flip the screen unintentionally, but this is standard Android behavior for apps supporting full rotation.....	14
1.7.4. Unit/Regression Testing Activities.....	14
Testing was performed using the Android Emulator with "Auto-Rotate" enabled.....	14
Test: Open a video -> Rotate emulator 180 degrees (Upside Down).....	14
Result: Application UI rotated 180 degrees, allowing the "charging port" to be at the top while the content remained upright for the viewer.....	14
1.7.5. Commit Link.....	14
https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/2c57f48f326b4daf719dc2caaa65ddeb14a1fd85	15
1.8. Comments region is not scrollable when comments are disabled in refactor branch (NewPipe) {#13248} [Bug].....	15
1.8.1. Description Of Changes.....	15
1.8.2. Pre/Post-Factoring Activities.....	15
1.8.3. Bug Localization.....	15
1.8.4. Change Impact Analysis.....	15
1.8.5. Unit/Regression Testing Activities.....	16
1.8.6. Commit Link.....	16
1.9. Add search to playlist append dialog (NewPipe) {#13154} [Feature].....	16
1.9.1. Description Of Changes.....	16
1.9.2. Pre/Post-Factoring Activities.....	16
1.9.3. Change Impact Analysis.....	17
1.9.4. Unit/Regression Testing Activities.....	17
1.9.5. Commit Link.....	17

1.10. Option to hide fully watched videos from History page {#13275}.....	17
1.10.1. Description of Changes.....	17
1.10.2. Pre/Post-Factoring Activities.....	18
1.10.3. Bug Localization.....	18
1.10.4. Change Impact Analysis.....	18
1.10.5. Unit/Regression Testing Activities.....	19
1.10.6. Commit Link.....	19
1.11. Incorrect scroll position on subscriptions page when returning from channel group {#13220}.....	19
1.11.1. Description of Changes.....	19
1.11.2. Pre/Post Factoring Activities.....	20
1.11.3. Bug Localization.....	20
1.11.4. Change Impact Analysis.....	20
1.11.5. Unit/Regression Testing Activities.....	21
1.11.6. Commit Link.....	21
1.12. Persistent repeat setting on restart (NewPipe) {#13325} [Enhancement].....	21
1.12.1. Description Of Changes.....	21
1.12.2. Pre/Post-Factoring Activities.....	21
1.12.3. Change Impact Analysis.....	21
1.12.4. Unit/Regression Testing Activities.....	21
1.12.5. Commit Link.....	21

1. Change Requests

1.1. Extracting Section Lists From Channel Tab Feature (NewPipe Extractor) {Self-Created} [Enhancement]

1.1.1. Description Of Changes

Currently the extractor only allows extracting items from the videos, playlist, and shorts tab on a youtube channel page. An enhancement to this extractor was to extract from the home (feature) tab as well. This was necessary to extract a list of lists such that it was possible to identify those lists and extract from them using another extractor. This required changing the TabExtractor, Creating a new RendererListInfoltemExtractor, RendererListInfoltemCollector, a RendererListInfoltem and making tiny edits in other files.

1.1.2. Pre/Post-Factoring Activities

In the extractor there is an existing test suite for the code base. Many of the tests that exist in the code base have been explicitly set to not run or actually fail, this is due to the extraction of outside services that may not contain the same structure it once had. After Implementation these tests would be run per identification of affected files.

1.1.3. Change Impact Analysis

Implementation of this enhancement would need to keep impact at 0% such that all extractors kept functionality. This was necessary as the enhanceable extractor was used in their other software newpipe which hosts all the data being extracted from the NewPipeExtractor. It contained a file such that the Info Items and tabs were being identified and would need further change in order to combat compilation errors.

1.1.4. Unit/Regression Testing Activities

Using the existing test suite creating unit tests using Test Driven Development was used. After implementation was done the tests were run multiple times before they all succeeded. Then it was time to move onto the other existing tests to make sure any files affected were run through those existing test suites to ensure previously functionality stayed the same.

1.1.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipeExtractor-TNT/commit/c7e7404ff66ce8acb5dbe404bf7cc06258d04740>

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/056ff2d3d7240e255c44023502fdf9695abe02ba>

1.2. Extracting Featured Channels (NewPipe Extractor) {1454} [Feature]

1.2.1. Description Of Changes

Using the Enhancement from the previous self created issue the implementation became possible. It uses the home (feature) tab extractor to extract the renderer list such that it contains the title of the featured channels, the type as a string (featured channels or not), the ListLinkHandler of the featured channels which is similar to the tab, and the index in the list of the tab extractor list. Using the existing ChannelInfoItems and its extractor and collector the ChannelListExtractor, ChannelInfoItem, Youtube implemented classes as possible. This would allow users to extract the channels that reside in grouped featured channels on the home (feature) tab of the channel page.

1.2.2. Pre/Post-Factoring Activities

Testing was done differently in this case as it would require a full new test suite setup. Features were implemented first and then unit tested later. Regression testing was still possible and ran after code implementation to consistently ensure functionality.

1.2.3. Change Impact Analysis

Because most of this is new features and wasn't touching much of the existing code it had little to no ripple effect. However it added an extra layer to most of the Infos for retrieving extractors and such so they can now extract ChannelListInfos. However currently they return null on such calls since they have not been implemented.

1.2.4. Unit/Regression Testing Activities

Creating unit tests for this became a little more simple as existing abstract test cases could be inherited from however functionality needed to match what was truly expected and needed. Creating test suites to pull from Linus Tech Tips channel and T-series channels as they both contained featured channels while also one having many and the other not as much. Testing failed states was important as well so that anyone using this extractor would get an expectation handler such that they could understand the problem occurring. This actually helped with introducing those exceptions.

1.2.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipeExtractor-TNT/commit/740992a9027c66a2ab3de7a88e5ad348a021dad6>

1.3. Exiting Channel Group Returns To Main page (NewPipe) {13221} [Bug]

1.3.1. Description Of Changes

This was a small change basically anytime a user pressed the back button in subscriptions navigation tab it went straight back home. To fix this I used a similar fix to how the search engine would be searched for in a stack of pages that have been visited and if the subscription page was in that stack it would go all the way back to it upon being pressed.

1.3.2. Pre/Post-Factoring Activities

Since this was a frontend activity there wasn't really any testing activities for this and as such debugging was used heavily. Similarly the app was tested personally making sure that functionality stayed the same that was changed.

1.3.3. Bug Localization

To identify the bug it was necessary to use the debugging feature. First I used static analysis to identify naming of functions or classes to identify the Stack for fragments. After this identifying the exact functionality that was adding the subscription fragment and the functionality of the stack was being dealt with on back press became difficult. Using breakpoints allowed a step by step process on add and then removal gave the location of how the pages were being handled on back. Unless the search fragment existed in the stack it would always go back to the home otherwise if the search fragment was in the stack it would go there first.

1.3.4. Change Impact Analysis

Since I added this fix this means it would need to not only check if the search exists in the stack but also the subscription fragment is in the stack. This could possibly affect load time however while testing the loading time on a back press there was no noticeable difference. Further testing could be made for exact time measurements however I deemed this unnecessary especially for how the repo deals with these tests.

1.3.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/bc23aac22329e40edf4fb143278751f62fefe2a1>

1.4. Add "Stop after current song ends" playback option (NewPipe) {#12726} [Feature]

1.4.1. Description Of Changes

The application lacked a feature to play the current track and stop immediately afterwards without clearing the rest of the queue. To implement this, a boolean flag `stopAfterCurrent` was added to the `PlayQueue` class, serving as the source of truth for the user's intent. A new menu item was added to `menu_play_queue.xml` to allow users to toggle this state. The `Player.java` controller logic was modified to intercept the `DISCONTINUITY_REASON_AUTO_TRANSITION` event. If the flag is set to true, the player pauses execution and resets the flag, rather than proceeding to the next track.

1.4.2. Pre/Post-Factoring Activities

Initial testing involved hardcoding the `stopAfterCurrent` flag to true within the `PlayQueue` constructor to verify the player interception logic before building the UI. It was discovered that `NewPipe` instantiates specific subclasses (like `SinglePlayQueue`), so the hardcoded check was moved to `Player.initPlayback()` for verification. Post-factoring involved removing these debug overrides and implementing the dynamic menu toggle in `PlayQueueActivity.java`.

1.4.3. Change Impact Analysis

The risk is minimal as the default state of the boolean is false, preserving the existing autoplay behavior for all users unless they explicitly opt-in via the menu. The change touches `PlayQueue.java` (Model), `Player.java` (Controller), and `PlayQueueActivity.java` (View), but adheres to the existing architecture.

1.4.4. Unit/Regression Testing Activities

Manual verification was performed using the Android Logcat with a custom debug tag (`481_DEBUG`).

Test Case 1: Enable toggle -> Allow song to finish -> Result: Player paused, Toast notification displayed "Stopping after this song."

Test Case 2: Enable toggle -> Disable toggle -> Allow song to finish -> Result: Player auto-transitioned to the next track.

Test Case 3: App restart -> Result: Flag successfully reset to false (due to transient keyword), preventing accidental stops in future sessions.

1.4.5. 1.5.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/884e1971d2950ec9ef0e4b9a549128d7d82d98f7>

1.5. Playlist isn't queued if the first video is unavailable (NewPipe) {#13226} [Bug]

1.5.1. Description Of Changes

A "Logic Fail" occurred where an entire playlist would fail to queue if the very first video (Index 0) was unavailable (e.g., region-locked). The UI would display a generic "Content Unavailable" error screen and abandon the queue creation. The fix involved modifying VideoDetailFragment.java. The openMainPlayer() method contained a guard clause that prevented the Player Service from starting if the metadata for the first video was null. This was bypassed to allow the player to launch if a valid queue (PlayQueue) existed. Additionally, the runWorker() error handler was updated to log the failure but force a call to openVideoPlayerAutoFullscreen(), handing control to the backend MediaSourceManager.

1.5.2. Pre/Post-Factoring Activities

Pre-Factoring: Investigated MediaSourceManager and PlaylistAppendTask assuming the issue was a backend failure.
Post-Factoring: Confirmed that the backend logic for skipping bad videos already existed; the UI was simply preventing it from executing.

1.5.3. Bug Localization

The bug was localized by reproducing the crash using the "Animal Crossing New Horizons OST" playlist, where the first video is unavailable. Logcat showed repeated calls to showSnackBarError() originating from VideoDetailFragment. Breakpoints revealed that openMainPlayer was never being called because the UI fragment treated the single video failure as a fatal error for the entire playlist context.

1.5.4. Change Impact Analysis

This change improves application robustness. It relies on the existing backend logic in MediaSourceManager to scrub unavailable items. There is a minor impact on user experience where the player may briefly show a "Loading" state for the failed video before skipping to the next valid index, but this is preferable to a total failure.

1.5.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/c279f528a85145ef27b08ff82ca88cef719bd5e0>

1.6. Allow upside-down portrait mode (reverse portrait) for video playback (NewPipe) {#12779} [Feature]

1.6.1. Description Of Changes

The application previously forced a strict "Portrait" orientation or "Landscape" orientation, making it difficult to watch vertical videos while charging the device (which usually requires the phone to be held upside-down). The fix involved updating the AndroidManifest.xml file. The android:screenOrientation attribute for the main activity was changed to "fullSensor". This instructs the Android OS to respect all four orientation degrees (0°, 90°, 180°, 270°) based on the accelerometer, enabling "Reverse Portrait" mode.

1.6.2. Pre/Post-Factoring Activities Pre-Factoring:

Tested on an Android 16 Emulator. Rotating the device 180 degrees resulted in either no change (UI upside down relative to user) or a forced rotation to Landscape. Post-Factoring: Verified that fullSensor correctly interprets the 180-degree rotation and flips the UI while maintaining the vertical layout.

1.6.3. Change Impact Analysis

This is a configuration-only change. It has zero impact on codebase logic or compilation time. The only potential side effect is if a user's device accelerometer is overly sensitive, it might flip the screen unintentionally, but this is standard Android behavior for apps supporting full rotation.

1.6.4. Unit/Regression Testing Activities

Testing was performed using the Android Emulator with "Auto-Rotate" enabled.

Test: Open a video -> Rotate emulator 180 degrees (Upside Down).

Result: Application UI rotated 180 degrees, allowing the "charging port" to be at the top while the content remained upright for the viewer.

1.6.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/2c57f48f326b4daf719dc2caaa65ddeb14a1fd85>

1.7. Comments region is not scrollable when comments are disabled in refactor branch (NewPipe) {#13248} [Bug]

1.7.1. Description Of Changes

When you select the video info tab, scroll to the bottom, then switch to the comments tab, it is expected that the user should be able to scroll back up to the video. However, when there are no comments, either due to the comments being disabled or no comments being posted, most of the area does not respond to any touch, except for the very top, which is not what is expected. This issue only occurs in the `refactor` branch, which is the branch which will eventually become the new main development branch. The change extends the invisible comment region vertically to fill the empty space so that it can be used as a pushoff target for scrolling

1.7.2. Pre/Post-Factoring Activities

The existing code was already written in well-composed Kotlin prior to the change. The new code was formatted to match the code styles of the existing codebase.

1.7.3. Bug Localization

The bug was localized by identifying a video with comments disabled ("Adventures Through Toy Story" by Disney Kids), switching to the video info tab, scrolling to the bottom, then switching to the comments tab. Then, the scrolling gesture was tested on different areas of the comments region. This demonstrated that it only responded if the scroll gesture was close enough to the "Comments are disabled" text/emoticon. Then the codebase was searched for the comment section code which was found in `CommentSection.kt`. The layout described in `CommentSection.kt` indicated that the `LazyColumn` used did not extend the full vertical height of the available space.

1.7.4. Change Impact Analysis

This is a UI change which is used for all comment sections on videos. Therefore, it impacts comment sections with disabled comments, no comments, and comments. The risk of breaking the layout of sections with comments is mitigated by manual testing. The change has no impact on the logic or functionality of the app beyond the comment section.

1.7.5. Unit/Regression Testing Activities

Since it is not possible to unit test this issue and there is no system testing framework in NewPipe, manual testing was done to ensure the bug was fixed and no regressions occurred. To do this, the unmodified app was tested on a virtual phone and virtual tablet to verify that it occurred. Then, the corrected app was tested on the same virtual phone and virtual tablet. The devices were tested in both vertical and horizontal orientations. Videos with disabled comments, no comments, and comments were all tested and verified to exhibit correct behavior regarding the scrolling ability, and regular functionality of the comment section with comments and replies was tested to ensure no regressions occurred.

1.7.6. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/e0a69e8840b0070acae00683e2ab436a53185805>

1.8. Add search to playlist append dialog (NewPipe) {#13154} [Feature]

1.8.1. Description Of Changes

Prior to the change, the only way to find the custom playlist that you want to add a video to was to scroll all the way through the list until you found it. This change adds a search box to the playlist append dialog which filters and sorts the playlists to make it easier to find the playlist you are looking for.

1.8.2. Pre/Post-Factoring Activities

Part of the purpose of the refactor branch is to convert the existing mixed Java and Kotlin codebase to only Kotlin. One of the files edited was a Java file, but I elected to only modify it and not convert it completely to Kotlin since a complete conversion would be out of the scope of the change and could introduce more issues. After originally implementing the change in Java and seeing it work properly, I decided to convert the new class I added, LocalPlaylistSearch, to Kotlin. This means that there will need to be less future work towards converting the code to Kotlin, and the new feature remains intact and functional.

1.8.3. Change Impact Analysis

This change impacts the UI for the playlist append dialog, since it adds a search box to the layout. This playlist append dialog is only used in one scenario, so there is low risk of any negative effects of the change. It does add a small amount of Java code to the codebase, which the refactor is aiming to eliminate, but this is unavoidable without converting the entire existing file to Kotlin, which is out of scope. New classes were written in Kotlin to avoid additional future maintenance work.

1.8.4. Unit/Regression Testing Activities

Unit tests were implemented to ensure the functionality and behavior of the sorting and filtering matched the expectations of how the sorting and filtering should operate, including in specific scenarios such as with multiple words or matches within words. This also included edge cases like no query and no entries.

1.8.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/96c85f20c60f60ade62a4c05b8172d810681c783>

1.9. Option to hide fully watched videos from History page {#13275}

1.9.1. Description of Changes

The History screen includes a “Fully Watched” filter intended to separate videos that have been completely viewed from those that were only partially watched. During development, some videos that had been watched to the end were still appearing as not fully watched.

The filtering logic in `StatisticsPlaylistFragment.java` was updated so that the application no longer requires the saved playback position to match the full duration exactly. A small tolerance threshold was added to account for minor differences between the stored playback progress and the actual end of the video.

This update was limited to the History filtering logic and did not require changes to the database schema or playback infrastructure.

1.9.2. Pre/Post-Factoring Activities

Pre-Factoring:

Before the change, videos that had been watched to completion were still appearing when the “Fully Watched” filter was disabled. This indicated

that the comparison used to determine whether a video counted as fully watched was too strict.

Post-Factoring:

After adjusting the comparison logic to include a small threshold, the filter behavior improved in some cases, but the issue was not fully resolved. Some videos were still being marked as incomplete even though they had been played to the end. This suggests that the remaining problem may be related to how playback progress is stored when a video finishes, rather than the filtering logic itself.

1.9.3. Bug Localization

The issue was isolated to the History filtering logic in `StatisticsPlaylistFragment.java`. This file handles how History entries are displayed and includes the logic that determines whether a video is considered fully watched.

The suspected problem was in the comparison between the saved playback position (`progressMillis`) and the full video duration (`durationSeconds * 1000`). This comparison was too strict, since playback progress often stops slightly before the exact end of the video.

1.9.4. Change Impact Analysis

This change only affects the filtering behavior on the History screen. It does not modify the playback engine, extractor logic, or database structure.

Its impact is limited to how the application interprets saved playback progress when deciding whether an item should appear under the current History filter settings. Because the update is contained within the filtering logic, the risk of affecting other parts of the application is low.

The main limitation is that if the stored playback progress is significantly lower than the actual end of the video, the filter may still classify a completed video as incomplete. In that situation, the underlying issue would likely need to be addressed in the playback progress tracking logic rather than in the History filter alone.

1.9.5. Unit/Regression Testing Activities

Formal unit tests were not added for this change. Testing was done manually in the Android Emulator because the issue depends on real playback behavior and interaction with the History screen. The process

involved launching the app, playing several short videos to completion, navigating to the History screen, toggling the “Fully Watched” filter, and observing which videos remained visible under each filter state.

This manual testing showed that the tolerance-based comparison improved the filter’s behavior in some situations, but the issue was not fully resolved. In certain cases, videos that had been played to the end still appeared as partially watched, likely because the saved playback position did not get close enough to the actual duration.

1.9.6. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/0b124ff9be3fcb3c29c882440dc9fde926414175>

1.10. Incorrect scroll position on subscriptions page when returning from channel group {#13220}

1.10.1. Description of Changes

This issue involved the Subscriptions page scrolling to an incorrect position after returning from a channel group view. When a user scrolled down on the Subscriptions page and then opened a channel group such as the “All” group, pressing the back button returned the user to the Subscriptions page, but the list appeared shifted downward instead of restoring the expected position.

The change introduces a state flag (`isLayoutSetup`) in `SubscriptionFragment.kt` to prevent the layout initialization method `setupInitialLayout()` from running multiple times during the fragment lifecycle. Previously, `setupInitialLayout()` was executed every time `initViews()` was called, which could rebuild parts of the `RecyclerView` structure and interfere with how the saved scroll state was restored when navigating back from a channel group.

1.10.2. Pre/Post Factoring Activities

Pre-Factoring:

This issue involved the Subscriptions page returning to an incorrect scroll position after navigating back from a channel group view. When a user scrolled down on the Subscriptions page and then opened a channel group such as the “All” group, pressing the back button returned the user

to the Subscriptions page, but the list appeared lower than expected instead of restoring the previous position.

Post-Factoring:

The update adds a state flag (`isLayoutSetup`) in `SubscriptionFragment.kt` to prevent the layout initialization method `setupInitialLayout()` from running multiple times during the fragment lifecycle. Previously, `setupInitialLayout()` was triggered every time `initViews()` was called, which could rebuild parts of the `RecyclerView` and interfere with restoring the saved scroll state when returning from a channel group.

1.10.3. Bug Localization

The bug was localized to the Subscriptions UI logic in `SubscriptionFragment.kt`, which manages the `RecyclerView` that displays subscription channels and feed groups. The investigation focused on the behavior of `initViews()`, which initializes the fragment's UI, `setupInitialLayout()`, which constructs the subscription list and feed group components, and `itemsListState`, which stores the `RecyclerView`'s scroll position. The issue appeared to occur because `setupInitialLayout()` could run multiple times when the fragment view was recreated, which could alter the layout structure before the saved scroll state was restored.

1.10.4. Change Impact Analysis

The modification only affects the Subscriptions fragment's UI initialization logic. No changes were made to the database layer, subscription data models, networking components, or navigation outside this fragment. The update is isolated to how the fragment sets up its layout, which keeps the risk of impacting other parts of the application low. The goal was to stabilize the layout initialization so the existing scroll-state restoration logic can work as intended.

1.10.5. Unit/Regression Testing Activities

This modification only affects the UI initialization logic within the Subscriptions fragment. It does not change the database layer, subscription data models, networking components, or any navigation behavior outside this fragment. The update is limited to how the fragment sets up its layout, which keeps the risk of unintended side effects low.

1.10.6. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/3ec2ae54da68976aaf44a06f2a11e24d34035190>

1.11. Persistent repeat setting on restart (NewPipe) {#13325} [Enhancement]

1.11.1. Description Of Changes

This change added a toggleable setting to allow the repeat and shuffle settings to persist across different sessions of the app.

1.11.2. Pre/Post-Factoring Activities

I took a look at Player.java and PlayerHelper.java, the main two classes which hold the functionality for the shuffle and repeat settings. I saw that there was a switch statement which could be simplified, so I simplified it. After completing the feature, I formatted the code to match the rest of the code better.

1.11.3. Change Impact Analysis

This change has moderate risk. It adds a setting (which are well containerized) and modifies core player code to change repeat and shuffle behavior. This risk was mitigated by manual testing.

1.11.4. Unit/Regression Testing Activities

No unit testing was possible at the time due to the nature of the Player class. However, it was manually tested with a virtual Android device.

1.11.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/7cdab496a8000d3660ddfc79e9ba2b581f0ce7d8>

1.12. Navigate up button returns from playlist with a bug {#13327}

1.12.1. Description of Changes

This issue was caused by the navigate-up button taking the user back to the search screen instead of returning them to the channel's Playlists tab. The problem showed up when someone searched for a channel, opened the channel page, switched to the Playlists tab, opened a playlist, and then pressed the navigate-up button. Instead of stepping back one screen, the app jumped all the way back to search.

The fix updates the navigation logic in MainActivity.java, inside onHomeButtonPressed(). Now, if the current screen is a PlaylistFragment, the app hands things off to the normal Android back navigation

(onBackPressed()), which correctly returns the user to the previous fragment in the stack.

1.12.2. Pre/Post-Factoring Activities

Pre-Factoring:

Before the change, pressing the navigate-up button from a playlist screen triggered the navigation helper logic, which tried to jump back to previously tagged fragments like the search screen. Because of this, the app skipped the expected intermediate step (the channel's Playlists tab) and went straight back to the search results.

Post-Factoring:

The update adds a check in MainActivity.java to see whether the current fragment is a PlaylistFragment. When it is, the navigate-up button now calls onBackPressed(), which follows the normal fragment back stack. This returns the user to the correct parent screen inside the channel instead of jumping back to search.

1.12.3. Change Impact Analysis

The issue came from the navigation logic in MainActivity.java, specifically in onHomeButtonPressed(), which controls how the toolbar's navigate-up button behaves. This method uses FragmentManager and NavigationHelper to decide which screen to show when the user presses the button.

The incorrect behavior happened because the method always tried to jump back to fragments like the search screen, without checking whether the current screen was a playlist view that should return to its parent. By checking the active fragment in the fragment_holder container and detecting when it's a PlaylistFragment, the navigation logic can hand things off to Android's normal back stack, which returns the user to the correct parent screen.

1.12.4. Unit/Regression Testing Activities

Testing focused on making sure the navigation flow behaved correctly after the update. The app was opened in the Android emulator and the same steps that originally caused the issue were repeated. A channel was searched for, the channel page was opened, the Playlists tab was selected, a playlist was opened, and the navigate-up button was pressed. After the change, the navigate-up button returned the user to the channel's Playlists tab instead of jumping back to the search screen. Basic checks also confirmed that normal back navigation still worked as

expected and that moving between search results, channels, and playlists continued to behave the same way.

1.12.5. Commit Link

<https://github.com/WSU-CptS-481-Spring-2026/NewPipe-TNT/commit/a8a05f5c31c230b4a2658760165edadf6991b9f2>